

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)**

**Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»**

**Лабораторная работа №2
по курсу «Операционные системы»**

**Выполнил: С. М. Закатин
Группа: М8О-203Б-25
Преподаватель: Е. С. Миронов**

Москва, 2026

Условие

Цель работы:

Приобретение практических навыков управления процессами в операционной системе и организации обмена данными между процессами посредством неименованных каналов.

Задание:

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними. Родительский процесс должен создать дочерний процесс. Открытый входной файл перенаправляется в стандартный поток ввода дочернего процесса, а стандартный поток вывода дочернего процесса - в неименованный канал. Родительский процесс читает полученные данные из канала и выводит их в свой стандартный поток вывода. Родительский и дочерний процессы должны быть представлены разными программами. Необходимо обрабатывать системные ошибки, которые могут возникнуть в процессе работы.

Вариант: 6

Во входном файле записаны целые числа, разделённые пробельными символами. Дочерний процесс вычисляет их сумму и выводит результат в стандартный поток вывода. Количество чисел может быть произвольным.

Метод решения

Программа разделена на два исполняемых файла: `parent` и `child`. Родительский процесс открывает входной файл, создаёт неименованный канал и запускает дочернюю программу. Перед вызовом `execv` стандартный ввод дочернего процесса связывается с дескриптором входного файла, а стандартный вывод - с записывающим концом канала при помощи `dup2`. После успешного запуска родитель закрывает неиспользуемые дескрипторы, читает результат из канала и печатает его в консоль.

Дочерняя программа читает перенаправленный стандартный ввод блоками по 4096 байт. Разбор выполняется конечным автоматом `ParserState`, поэтому размер файла и количество чисел не ограничены размером одного буфера. Автомат учитывает знак числа, накапливает цифры текущего числа и добавляет завершённое значение к общей сумме при встрече разделителя. После достижения конца файла отдельно обрабатывается последнее число.

Схема обмена данными имеет следующий вид:

```
Файл -> stdin child -> вычисление суммы -> pipe -> parent -> stdout
```

Описание программы

Основные файлы проекта:

- `parent.c` - открытие файла, создание канала, запуск дочернего процесса и вывод результата;
- `child.c` - потоковый разбор входных данных и вычисление суммы;
- `os.h`, `os.c` - общий интерфейс для работы с файлами, процессами и каналами;
- `posix/process.c` - реализация создания процесса, перенаправления стандартных потоков, запуска программы и ожидания её завершения;
- `posix/pipe.c`, `posix/file.c` - обёртки над системными функциями для каналов и файлов;

- `auto.h`, `dtors.h` - реализация RAII-подобных типов для автоматического освобождения ресурсов;
- `.github/workflows/sanitize.yml`, `win.yml` - сценарии непрерывной интеграции для автоматической сборки и проверки проекта в Linux и Windows;
- `CMakeLists.txt` - описание сборки программ `parent` и `child`.

Структура `ParserState` хранит текущее значение числа, его знак, признаки наличия цифр и знака, количество распознанных чисел и накопленную сумму. Функция `parser_init` инициализирует состояние анализатора, функция `parser_process_char` обрабатывает очередной символ, а `parser_flush` завершает текущее число и добавляет его к сумме.

Хотя язык Си не предоставляет RAII на уровне языка, в платформенном слое проекта реализован близкий механизм на основе атрибута `cleanup`, поддерживаемого компиляторами GCC и Clang. Макрос `DEFINE_CLEANUP_TYPE` создаёт тип-обёртку, функцию очистки и операцию передачи владения ресурсом. Например, переменная, объявленная как `AUTO(AutoFile)`, автоматически вызывает `close` при выходе из области видимости, в том числе при досрочном возврате из функции. Это упрощает обработку ошибок и защищает программу от утечек файловых дескрипторов.

Макрос `move_out` используется, когда владение дескриптором необходимо передать другому участку программы. Он возвращает сохранённое значение и одновременно заменяет его в исходной RAII-переменной на `OS_INVALID_HANDLE`. Благодаря этому функция автоматической очистки не закрывает уже переданный ресурс повторно. В проекте данный приём применяется при создании канала и при распределении его концов между родительским и дочерним процессами. Кроме `AutoFile`, предусмотрены типы `AutoString`, `AutoStrVec` и `AutoProcInfo` для автоматического освобождения строк, векторов строк и описания процесса.

Для работы с процессами и межпроцессного обмена используются следующие системные вызовы Linux:

- `open` - открытие входного файла на чтение;
- `pipe` - создание неименованного канала;
- `fork` - создание дочернего процесса;
- `dup2` - перенаправление `stdin` и `stdout` дочернего процесса;
- `execv` - замена образа дочернего процесса программой `child`;
- `read`, `write` - чтение файла и передача результата через канал;
- `close` - закрытие неиспользуемых файловых дескрипторов;
- `waitpid` - ожидание завершения дочернего процесса.

После каждого потенциально неуспешного действия проверяется возвращаемое значение. При ошибке выводится диагностическое сообщение, освобождаются уже полученные ресурсы, а программа завершается с кодом `EXIT_FAILURE`.

Для проекта настроена непрерывная интеграция GitHub Actions. Workflow `sanitize.yml` запускается при каждом `push` и создании или обновлении `pull request`. Он содержит независимые задания для Linux и Windows, поэтому изменения одновременно проверяются с разными реализациями платформенного слоя. Дополнительный workflow `win.yml` при каждом `push` конфигурирует проект через CMake, собирает программы и выполняет функциональный запуск в среде Windows.

Результаты

Проект был собран компилятором GCC 15.2.1 при помощи CMake в конфигурации Release. Сборка обеих программ завершилась успешно.

Для проверки использован входной файл следующего содержания:

```
10 20 30
-5 7 8
100 -25 5 0
```

В файле содержится 10 чисел, их ожидаемая сумма равна 150. Запуск выполнялся следующей командой:

```
./bin/parent test-data.txt ./bin/child
```

Программа вывела:

```
Sum of 3 numbers: 60
Sum of 3 numbers: 10
Sum of 4 numbers: 80
```

Фактический результат совпал с ожидаемым. В тесте присутствуют положительные, отрицательные и нулевые значения, а данные размещены в нескольких строках. Чтение блоками обеспечивает работу с произвольным количеством чисел без предварительной загрузки всего файла в память.

Для подтверждения фактического использования системных вызовов программа была запущена под управлением утилиты strace. Ниже приведён сокращённый фрагмент трассировки (служебные вызовы динамического загрузчика опущены):

```
openat(AT_FDCWD, "test-data.txt", O_RDONLY) = 3
pipe2([4, 5], 0) = 0
clone(..., SIGCHLD, ...) = 295021
dup2(3, 0) = 0
dup2(5, 1) = 1
execve("./bin/child", ["./bin/child"], ...) = 0
read(0, "10 20 30\n-5 7 8\n100 -25 5 0\n", 4096) = 28
read(4, "\0\0\0\0\0\0\0\0\n\0\0\0\0\0\0\0\0", 16) = 16
write(1, "Sum of 3 numbers: 60\n", 21) = 21
read(4, "\0\0\0\0\0\0\0\0\n\0\0\0\0\0\0\0\0", 16) = 16
write(1, "Sum of 3 numbers: 10\n", 21) = 21
read(4, "\0\0\0\0\0\0\0\0\n\0\0\0\0\0\0\0\0", 16) = 16
write(1, "Sum of 4 numbers: 80\n", 21) = 21

wait4(295021, ..., 0, NULL) = 295021
```

Трассировка подтверждает открытие файла, создание канала, создание дочернего процесса, перенаправление файловых дескрипторов, запуск отдельной программы child, передачу результата через канал и ожидание завершения дочернего процесса.

Дополнительно была исследована производительность программы при помощи утилиты perf. Входной файл big_data.txt размером 2,85 Мбайт содержал 650,057 целых чисел, распределённых по 100,000 строкам-командам. Для исключения миграций между производительными и энергоэффективными ядрами программа была закреплена за одним Р-ядром. Вывод 100 тысяч результатов перенаправлялся в /dev/null, чтобы измерение характеризовало обработку данных и межпроцессный обмен, а не скорость отрисовки терминала. Статистика собиралась по результатам 30 запусков:

```
taskset -c 2 perf stat -r 30 \
```

```
-e task-clock,context-switches,cpu-migrations,page-faults,cycles,instructions,
  branches,branch-misses \
./build/bin/parent big_data.txt ./build/bin/child > /dev/null
```

Основные результаты измерения представлены в таблице 1.

Показатель	Среднее значение
Полное время выполнения	55,26 мс
task-clock	54,91 мс
Переключения контекста	593
Миграции между ядрами	0
Страничные ошибки	140
Такты процессора	122 683 532
Выполненные инструкции	245 304 278
Переходы	57 746 712
Ошибочные предсказания переходов	611 754

Таблица 1 — Результаты измерения производительности

Среднее полное время выполнения составило 55,26 мс с отклонением 0,10%. Производительность достигла приблизительно 11,76 млн чисел или 1,81 млн строк-команд в секунду, что соответствует примерно 85 нс на одно число. Отношение числа выполненных инструкций к числу тактов равно $245\,304\,278 / 122\,683\,532 \approx 2,00$ инструкции за такт. Доля ошибочных предсказаний переходов составила $611\,754 / 57\,746\,712 \approx 1,06\%$.

Нулевое число миграций и малый разброс результатов показывают, что закрепление за одним ядром сделало измерение воспроизводимым. За запуск происходило в среднем 593 переключения контекста, что связано с построчной передачей результатов между двумя процессами через канал. События `cpu_atom` не регистрировались, поскольку программа выполнялась на производительном ядре. В полное время входят не только вычисления дочернего процесса, но и открытие файла, создание канала, вызовы `fork` и `exec`, загрузка программы, передача 100 тысяч результатов и ожидание завершения процесса.

При выводе результатов непосредственно в терминал полное время составляло около 1,575 с. Перенаправление вывода в `/dev/null` уменьшило его приблизительно в 28,5 раза. Следовательно, главным ограничением интерактивного сценария является вывод большого количества строк, тогда как вычисление сумм и обмен сообщениями выполняются существенно быстрее.

Помимо ручного запуска, проект проверяется автоматически средствами CI. В Linux workflow собирает проект с AddressSanitizer. Используются флаги `-fsanitize=address`, `-fno-omit-frame-pointer`, `-O1` и `-g`. Затем программы запускаются на тестовом файле. AddressSanitizer позволяет обнаружить выход за границы выделенной памяти, обращение к уже освобождённым объектам, двойное освобождение и ряд других ошибок работы с памятью; обнаруженная ошибка приводит к падению шага CI.

Вторая Linux-сборка создаётся в конфигурации Debug и запускается под управлением Valgrind с параметрами `-leak-check=full` и `-error-exitcode=1`. Первый параметр включает подробный поиск утечек памяти, а второй преобразует найденную ошибку в ненулевой код возврата, благодаря чему GitHub Actions помечает проверку как неуспешную. Совместное применение AddressSanitizer и Valgrind полезно, поскольку инструменты используют разные механизмы анализа и частично дополняют друг друга.

В Windows workflow собирает проект компилятором MSVC с параметрами `/analyze` и `/WX`. Статический анализатор проверяет потенциально опасные конструкции без выполнения программы, а `/WX` превращает предупреждения в ошибки сборки. После этого выполняется функциональный запуск `parent.exe` и `child.exe` на подготовленном входном файле. Таким образом, CI проверяет не только компилируемость проекта на двух платформах, но и базовый сценарий межпроцессного взаимодействия и корректность управления памятью.

Выводы

В ходе лабораторной работы была реализована не просто передача значения между двумя программами, а полноценная цепочка обработки данных: файл становится стандартным вводом дочернего процесса, вычисленный результат поступает в канал через стандартный вывод, после чего родительский процесс читает канал и отображает результат пользователю. Такое решение показывает, что перенаправление файловых дескрипторов позволяет связывать независимые программы без изменения логики их ввода и вывода.

На практике особенно важным оказалось правильное управление временем жизни дескрипторов. После запуска дочерней программы родитель закрывает исходный файл и записывающий конец канала, оставляя только дескриптор для чтения. Благодаря этому достижение конца потока корректно определяется вызовом `read`; если бы хотя бы одна лишняя копия записывающего дескриптора осталась открытой, родительский процесс мог бы бесконечно ожидать новые данные. Вызов `waitpid` дополнительно обеспечивает корректное завершение взаимодействия и не допускает появления процесса-зомби.

Потоковый разбор в программе `child` не зависит от границ блоков чтения: состояние незавершённого числа сохраняется в структуре `ParserState`, поэтому число может оказаться разделено между двумя вызовами `read`. Такой подход масштабируется на файлы произвольного размера и требует постоянного объёма памяти. Разделение основной логики и платформенного слоя `os` также сделало работу с процессами, файлами и каналами более понятной и позволило локализовать обработку системных ошибок.

Полезным результатом работы стало применение RAII-подобного управления ресурсами в языке Си. Тип `AutoFile` связывает файловый дескриптор с функцией очистки, а `move_out` явно выражает передачу владения и предотвращает двойное закрытие. Поэтому временные дескрипторы корректно освобождаются даже на ветвях с ошибками, где ручное управление ресурсами обычно приводит к дублированию кода. Это показало, что идеи RAII применимы не только в C++, но и в системной программе на Си при использовании возможностей компилятора и строгого соглашения о владении ресурсами.

Автоматизация проверок в GitHub Actions дополнила ручное тестирование. Обычная сборка подтверждает только синтаксическую и функциональную корректность, тогда как запуск с `AddressSanitizer` и `Valgrind` позволяет обнаруживать скрытые ошибки времени выполнения и утечки ресурсов. Параллельная проверка Linux- и Windows-реализаций снижает риск того, что изменение общего интерфейса `os` нарушит только одну из поддерживаемых платформ. В результате workflow выступают воспроизводимой частью проверки программы: один и тот же набор действий автоматически выполняется для каждого изменения кода.

Исходная программа

```
1 #include <stdio.h>
2 #include <string.h>
3
4 #include "../os/os.h"
5
6 int main(int argc, char *argv[]) {
7     char filename[256];
8     char childname[256];
9     if (argc == 2) {
10         fprintf(stderr, "Usage: %s <data_file> <child_executable>\n", argv[0]);
11         fprintf(stderr, "Example: %s data.txt ./bin/child\n", argv[0]);
12         return EXIT_FAILURE;
13     }
14     if (argc < 2) {
15         printf("Enter filename: ");
16         if (fgets(filename, sizeof(filename), stdin) == NULL) {
17             fprintf(stderr, "Failed to read file name from stdin\n");
18             return EXIT_FAILURE;
19         }
20
21         printf("Enter path to the child process: ");
22         if (fgets(childname, sizeof(childname), stdin) == NULL) {
23             fprintf(stderr, "Failed to read child path from stdin\n");
24             return EXIT_FAILURE;
25         }
26     } else {
27         strncpy(filename, argv[1], sizeof(filename) - 1);
28         strncpy(childname, argv[2], sizeof(childname) - 1);
29     }
30     filename[sizeof(filename) - 1] = '\0';
31     childname[sizeof(childname) - 1] = '\0';
32
33     os_file_handle_t file = os_file_open(filename, OS_FILE_READ);
34     if (file == OS_INVALID_HANDLE) {
35         perror("Failed to open file");
36         return EXIT_FAILURE;
37     }
38
39     os_file_handle_t r_pipe = OS_INVALID_HANDLE;
40     os_file_handle_t w_pipe = OS_INVALID_HANDLE;
41     if (os_pipe_create(&r_pipe, &w_pipe) < 0) {
42         perror("Failed to create pipe");
43         os_file_close(file);
44         return EXIT_FAILURE;
45     }
46
47     os_process_info_t info;
48     os_process_info_init(&info);
49     info.cmd = childname;
50     os_proc_info_add_arg(&info, childname);
51
52     info.redirect_stdin = file;
53     info.redirect_stdout = w_pipe;
54     os_process_handle_t pid = os_process_create(&info);
55     if (pid == OS_INVALID_HANDLE) {
56         perror("Failed to create child process");
```

```

57     os_file_close(file);
58     os_file_close(r_pipe);
59     os_file_close(w_pipe);
60     os_process_info_destroy(&info);
61     return EXIT_FAILURE;
62 }
63 os_file_close(file);
64 os_file_close(w_pipe);
65
66 char buffer[4096];
67 os_ssize_t bytes_read;
68
69 printf("Result from child: ");
70 while ((bytes_read = os_pipe_read(r_pipe, buffer, sizeof(buffer) - 1)) > 0) {
71     buffer[bytes_read] = '\0';
72     printf("%s", buffer);
73 }
74 printf("\n");
75
76 if (bytes_read < 0) {
77     perror("Failed to read from pipe");
78 }
79
80 os_process_close(pid);
81 os_file_close(r_pipe);
82 os_process_info_destroy(&info);
83
84 return EXIT_SUCCESS;
85 }

```

Листинг 1: Исходный код родительского процесса

```

1  #include <stdio.h>
2  #include <ctype.h>
3  #include <stdbool.h>
4  #include <stdlib.h>
5  #include "../os/os.h"
6
7  #define OS_STDIN_FILENO ((os_file_handle_t)0)
8  #define READ_BUFFER_SIZE 4096
9
10 typedef struct {
11     int value;
12     int sign;
13     bool has_digits;
14     bool has_sign;
15     int count;
16     int sum;
17 } ParserState;
18
19 static void parser_init(ParserState *state) {
20     state->value = 0;
21     state->sign = 1;
22     state->has_digits = false;
23     state->has_sign = false;
24     state->count = 0;
25     state->sum = 0;

```

```

26 }
27
28 static void parser_flush(ParserState *state) {
29     if (state->has_digits) {
30         state->sum += state->value * state->sign;
31         state->count++;
32     }
33     state->value = 0;
34     state->sign = 1;
35     state->has_digits = false;
36     state->has_sign = false;
37 }
38
39 static void parser_process_char(ParserState *state, char ch) {
40     if (isspace((unsigned char)ch)) {
41         parser_flush(state);
42         return;
43     }
44
45     if ((ch == '-' || ch == '+') && !state->has_digits && !state->has_sign) {
46         if (ch == '-') state->sign = -1;
47         state->has_sign = true;
48         return;
49     }
50
51     if (isdigit((unsigned char)ch)) {
52         state->value = state->value * 10 + (ch - '0');
53         state->has_digits = true;
54         return;
55     }
56
57     parser_flush(state);
58 }
59
60 int main(void) {
61     char buffer[READ_BUFFER_SIZE];
62
63     ParserState state;
64     parser_init(&state);
65
66     os_ssize_t bytes_read;
67
68     while ((bytes_read = os_file_read(OS_STDIN_FILENO, buffer, sizeof(buffer))) > 0) {
69         for (os_ssize_t i = 0; i < bytes_read; i++) {
70             parser_process_char(&state, buffer[i]);
71         }
72     }
73
74     parser_flush(&state);
75
76     if (bytes_read < 0) {
77         perror("Failed to read from stdin");
78         return EXIT_FAILURE;
79     }
80
81     printf("Sum of %d numbers: %d\n", state.count, state.sum);
82     return EXIT_SUCCESS;

```

Листинг 2: Исходный код дочернего процесса